

J.H. Fletcher — Onsite Demo Plan

Companion to [Pain Points and Demo Ideas.md](#). This one is the build list: what to construct before the onsite (including mocked-data POC applications), and what can be run live in the room.

Sources: the five call transcripts, Jordan's pain-point summary, and [jhletcher.com](#).

0. On Jordan's summary

It matches the transcripts closely and adds useful framing — particularly the through-line that the pain is concrete rather than abstract. Two notes before we build on it.

⚠ One number is inverted, and it changes the pitch

Jordan's item 3 says the backlog is *"roughly 70% fixes to existing functionality vs. 30% new/improvements, i.e., mostly firefighting."* The transcript says the opposite, and Jordan asked the clarifying question himself:

Jordan: *"How much in your development backlog is focused on making improvements in the existing system and adding new functionality versus how much of it is just fixes to existing functionality?"* **Mike:** *"Probably 70, 30."* **Jordan:** *"70% being which one, the first one?"* **Mike:** *"One, and making improvements. 30% on the fixes."*

— Discovery call, 24:29–25:10

So it's **70% improvements / 30% fixes**.

This is worth correcting because the inverted version undersells us. "Mostly firefighting" says they need stability. What Mike actually described is **a large improvement backlog they cannot get to** — which is precisely the throughput problem CoderFlow addresses. The real story is better: *they know what they want to build, they just can't build it fast enough with 1.5 developers*. Firefighting is the smaller half.

One addition worth making

Jordan's list is all internal-IT pain. Everything in §1 below — the revenue-side consequences of that pain — is missing from it, and it's the half that gets a CEO and a CFO to care.

1. What Fletcher actually does, and why it changes the demo

J.H. Fletcher & Co. — Huntington, WV, founded 1937. The world's leading manufacturer of **custom-built underground mining equipment**, principally roof bolters. ~300 employees, **more than two-thirds in production/warehouse**.

Product lines: roof bolters (coal and metal/non-metal), drill jumbos, longwall/utility bolters, mobile roof supports, degasification drills, powder loaders, prime mover tractors, scaling machines, utility vehicles, specialty equipment. Plus an **Engineered Solutions** division doing non-mining work (grit blasters, safe ladle access, sand spreaders), and **Cannon Mining**, acquired 2022.

Aftermarket and service: Fletcher Service Inc. (field service, machine audits, maintenance programs), a **parts/partials** business, **full and partial modernization, trade-in/exchange of core equipment**,

warranty claims, operator **simulators** via SimForMotion, and a **global distributor network**. Recent expansion into India.

The five business characteristics that should drive every demo

1. They are engineer-to-order, at extreme variance. Fletcher's own materials: machines are *"custom engineered and manufactured to meet customer specifications,"* and as a result *"many different models, design variations and options are now in use."* Eighty-eight years of custom builds.

This is the missing context for Julia's duplicate-part-number complaint. In a catalogue manufacturer, duplicate parts are untidy. In an **ETO shop with 88 years of one-off configurations**, the item master *is* the product knowledge. Duplicates there don't just pollute a table — they corrupt the ability to quote and fulfil parts for machines still underground. Her "too onerous to look it up" is an engineer rationally routing around a broken lookup.

2. Aftermarket is a long-tail revenue engine, and it runs on email. Parts ordering today is a web quote form, [jhffpartsdept@jhffletcher.com](mailto:jhfpartsdept@jhffletcher.com), and an after-hours phone line. There's no visible online parts lookup or customer portal. Machines last decades in a punishing environment.

Every parts quote is someone identifying a component for a custom machine built possibly decades ago. That is a **search problem over exactly the data they've told us is dirty**. And downtime on a roof bolter idles a whole mine section — turnaround time is the product.

3. Modernization requires per-serial-number lifecycle records. Full modernization restores a machine to *"look and operate like a brand new machine, complete with current certifications and approvals,"* with new warranty. Partial modernization targets drills and booms to *"lengthen the interval to whole machine modernization."*

That's an **installed-base intelligence** business: which machines, at which customers, at what age and duty, are due for partial vs. full modernization. If that's not queryable, modernization revenue is being left in the ground. This is the demo a CFO leans forward for.

4. They're in a safety-critical, MSHA-regulated industry. Fletcher invented and patented aspects of the ATRS (automated temporary roof support) and the first internal dust collection system. MSHA approval gates new machines and concepts. Owners *"should never make a modification"* — proposed design changes must be submitted to Fletcher engineering for review. Modernization issues **new certification tags**.

Certification and approval records are compliance evidence with legal weight. Julia's *"there is data we're afraid to lose"* almost certainly includes this. It also means engineering change control is a real, auditable business process — not an internal nicety.

5. Their PLM and their ERP don't talk. Ray raised **Teamcenter** himself as a webhook candidate: *"we could potentially create that linkage between the 400 and Team Center."*

Engineering BOMs and CAD live in Teamcenter; manufacturing and financials live on the IBM i. For an ETO manufacturer that gap is where cost, lead time and error accumulate — and it's the technical substrate under Julia's *"sales completes the purchase order and it goes automatically to engineering and generates a BOM."*

The reframe this buys us

The last demo showed **Work with Customers** — a generic file, with stub fleet data Ray noticed was invented. Fletcher's business is roof bolters, serial numbers, configurations, parts and modernization.

Every hour spent making the demo look like Fletcher's business is worth more than any additional CoderFlow feature. It converts "impressive tool" into "they understand us," which is the specific gap that produced suspicion last time.

Also note: Gary's improvised "fleet in service" panel with serial numbers and service dates was, by accident, exactly right for this business. That instinct should now be built properly on real structure.

2. How to use the two tiers

Tier A — build ahead. Mocked-data POC applications. These carry the vision. **Tier B — run live.** Short, real, unrehearsed-looking. These carry the credibility.

They do different jobs and the mix matters:

A polished mock proves *we understand your business*. A live run against *their* code proves *the platform is real*.

Last time we were heavy on the second with generic data, and it read as a magic trick. The fix is **one clearly-labelled mock of their business, plus one live run against their own code, side by side**. Never blur them — say plainly which is which. Ray already caught invented data once; being the ones who volunteer "this part is mocked" is what buys trust for everything we don't caveat.

3. Tier A — build before the onsite

Effort is my estimate for one engineer with CoderFlow, assuming mocked data. **"Needs"** is what we must get from Fletcher *now* — several of these have a lead time we don't control.

A1. Parts Finder — "which part fits serial #4417?" ★ top pick

Effort: M (3-5 days) · Needs: nothing (fully mockable); far better with a real item-master extract

A tablet-and-desktop EJS app for a parts rep or a distributor. Enter a customer or machine serial → machine configuration (model, build year, options fitted) → browse or search the assembly → part detail with supersessions, alternates, stock and lead time → add to quote → email the quote.

Two features carry the whole message:

- **Duplicate/near-duplicate detection**, inline: *"3 similar items exist — HYD-PUMP-A, HYDPUMP-A2, PUMP HYD A. Did you mean one of these?"* Informational, never blocking.
- **Fitment by serial number**, so the rep sees only parts valid for *that* machine as built.

Hits: Julia's duplicate part numbers; "people circumvent the system because they can" (this makes the correct path the *easy* path, which is the only guardrail that survives contact with users); the Mike reporting bottleneck; the aftermarket revenue story for Chuck and Rod.

Why it lands: it's their actual daily commercial motion, and it reframes data quality from an IT hygiene problem into a revenue problem. Best single artifact for the "we understand your business" gap.

A2. Installed Base / Fleet 360 ★ top pick

Effort: M (3-4 days) · **Needs:** nothing to mock

Customer → every machine they own (serial, model, build date, operating hours, mine site) → service history → warranty status → parts consumption → **modernization candidacy scoring** ("6 machines past the partial-modernization interval — est. \$X pipeline") → open issues.

Add the AI insight panel Gary already demoed, but grounded in real structure: *"This customer runs 14 Fletcher machines across 3 sites. Two J352s are approaching full-modernization age. Parts spend is down 22% year over year — possible service gap."*

Hits: de-Odom-ifying reporting; Julia's "visualizing our data is a major problem"; modernization revenue for Chuck and Rod.

Why it lands: it's the natural, correct version of what Gary improvised. It shows **revenue we can help them find**, which is the only demo language a CEO leaning toward Infor will weigh against sunk commitment.

A3. Order → Engineering → BOM workflow (Julia's exact request)

Effort: M-L (4-6 days) · **Needs:** rough sketch of their real order flow (ask Mike/Ray)

Build her stated example end to end, in Fletcher's ETO context: configured machine order completed in sales → webhook fires → **preliminary BOM generated from the configuration** → routed to an engineering review queue with a diff against the nearest previous build → engineer approves/adjusts → released to production scheduling → notification to purchasing. Include a **stubbed Teamcenter hook** so Ray sees his own idea implemented.

Hits: Julia's #1 automation ask; cross-functional handoff fragility (Jordan's item 7); Ray's Teamcenter idea.

Why it lands: this is the direct rebuttal to *"what an ERP like Infor gives you is a built-in process map."* We can't out-feature Infor's process library — but we can show **their** handoff, built to **their** process, in days. Julia has already said a generic best-practice map gets ignored.

A4. Production Schedule Board — retire the sticky notes

Effort: M (3-5 days) · **Needs:** nothing to mock; ideally a real work-order extract

A visual board replacing the physical sticky-note schedule: machine builds in progress by work centre, shop capacity, material-shortage flags, promised vs. projected ship dates. Include a **data-confidence panel** that flags where the schedule can't be trusted because the underlying data is bad.

Hits: the sticky-note schedule; *"we're getting into actually using the data in the system to plug into scheduling applications and we're finding a lot of database issues."*

Why it lands: it's the single most visible manual process in the building, two-thirds of the company works in production, and Mike raised it unprompted. The data-confidence panel is the honest touch — it says *we know this is blocked on data quality, and here's exactly what's blocking it*, which is far more credible than a board that pretends the data is clean.

A5. Field Service / Machine Audit — tablet app

Effort: M (3-4 days) · Needs: their tablet model and browser; one real audit checklist if possible

Fletcher Service Inc. does machine audits. A technician-facing tablet app: pick machine by serial or **scan** its tag → guided audit checklist → photo capture → wear measurements → auto-generated findings report → one-tap parts recommendation into a quote.

Hits: the unresolved scanner/tablet problem (§A6); mobile UI — *the Profound UI value Mike himself argued for*; field-service-to-parts revenue conversion.

Why it lands: it's a genuinely modern application on a device their people already carry, and it puts the demo where two-thirds of the workforce actually is. It also gives Mike his own argument back as a working artifact rather than a proposal.

A6. Warehouse scan flow — retire the landmine

Effort: S-M (2-3 days) · Needs: ⚠ their tablet hardware, a sample label, and the RPG scan program

Not really a demo — a liability we should close. Their tablets scan labels whose values carry prefixes that RPG parses. Through Genie ~3 years ago *"the RPG program didn't get everything the scanner put in."* It later started working unexplained, and **was never re-tested**.

Rebuild one scan flow (receiving or pick confirmation) as a responsive web screen and **prove the full scanner payload arrives**, on their hardware.

Why it lands: it's their only lived, concrete technical failure with our product. Mike will raise it. Arriving with evidence rather than reassurance is worth more than any new capability — and this has the longest lead time on the list, because it needs their physical hardware.

A7. Duplicate Part Number Analysis — real data, not mocked ★ pair with A1

Effort: S-M (2-3 days once we have data) · Needs: ⚠ an item-master extract (read-only)

Fuzzy-match their **actual** item master. Deliver: duplicate/near-duplicate clusters with counts, an estimated scale of the problem, blast-radius analysis per cluster (which BOMs, orders, transactions reference each), and a **safe merge plan** that sequences foreign-key updates so nothing is orphaned.

Why it lands: this is the highest-credibility artifact available to us, because it's **their data saying it, not us**. It quantifies Julia's top process complaint with a number she's never had. Pair it with A1: *here's how bad it is today, and here's the screen that stops it getting worse*.

This is also the strongest reason to chase a data extract **now**. Everything else can be mocked. This one cannot, and it's the one that will change the room.

A8. Executive dashboard

Effort: S-M (2-3 days) · Needs: nothing to mock

Bookings, backlog, shop load, on-time delivery, parts revenue, modernization pipeline, fleet by customer. Built for Julia, Chuck and Rod rather than for IT.

Hits: *"visualizing our data is a major problem within the company"*; the Odom reporting bottleneck.

Why it lands: Julia named data visualisation as an explicit pilot goal, and this is the one artifact the non-technical executives can evaluate on their own judgement without translation.

A9. MSHA certification & machine-records dossier

Effort: S-M (2-3 days) · Needs: confirmation of how they track certs today

Per serial number: approval numbers, certification tags, manual revisions, engineering change requests, modernization history, warranty events. Exportable as a compliance dossier.

Hits: *"there is data we're afraid to lose"*; the design-change review process; modernization's "new certification tags."

Why it lands: regulated-industry record-keeping is a risk story, and risk is the language Rod and Chuck are already using to justify Infor. This turns it around: **the records are safer curated on a system you control than migrated into a package.**

A10. Warranty claim intake & triage

Effort: S (1-2 days) · Needs: nothing to mock

Claim submitted → validated against machine serial and warranty terms → duplicate-claim check → routed → notification → status visible to the customer.

Hits: cross-functional handoff fragility; an existing form-and-email process.

A11. Distributor / dealer portal concept

Effort: M (3-4 days) · Needs: nothing to mock

Fletcher sells through a *"global network of distributors."* A branded external portal: order status, parts lookup and quoting, warranty submission, machine records for their territory.

Why it lands: it's **new revenue-side capability**, not modernization of something old. It answers "what could we build that we can't today?" — and it's exactly the kind of thing an ERP package will not give them and will charge heavily to bolt on. Speculative, so pitch it as a direction rather than a plan.

A12. Cannon Mining catalogue reconciliation (*hypothesis — verify first*)

Effort: S-M · Needs:  confirmation this is even a real problem

They acquired **Cannon Mining in 2022**. Acquisitions almost always arrive with a separate part numbering scheme, duplicate suppliers and a parallel BOM structure.

Ask before building: *"How is Cannon's product data held relative to Fletcher's?"* If they're separate, this is a sharp, specific pain we identified without being told — which is a strong signal of homework. **If they're already integrated, drop it.** Don't assert this one in the room until confirmed.

4. Tier B — run live on the day

Short, real, against **their** code and data. Each should run in well under ten minutes. Ideally launched early and reviewed later, the way Gary paced the last demo.

Prerequisite for almost all of these: their source in a repo, and read-only DB access. This is the single biggest scheduling risk in the whole plan — Gary already flagged that getting legacy sources into a repo is the real setup effort. **Start it now, not the week before.**

Analysis and discovery — read-only, zero risk, easiest to get consent for

B1. Code review + tech debt epic on a real Fletcher program. What Gary demoed, but on their code. Point it at a genuinely gnarly legacy program → epic + tickets, labelled low-hanging-fruit vs. breaking-change, with reasoning. (*For Mike and Ray.*)

B2. Documentation generation — technical + user guide. Run it on **the three-year-bug program** or an RPG II module. Then hand Mike the red pen and ask him to find what it got wrong. (*See [Pain Points §6.B](#) — the single highest-leverage move available.*)

B3. Dead code and usage map. Object last-used dates, journal analysis, what programs touch what files. Directly answers *"there's not a really good way to track what is and isn't being used."* **Say out loud that this is valuable to them even if they choose Infor** — scoping an ERP migration needs exactly this inventory, and they can't produce it today.

B4. Database health scan. — ★ *strongest live analysis demo.* Invalid numerics in program-described files, orphaned records, broken referential integrity, impossible dates, fields that are supposed to be codes and aren't. Mike's exact reported pain — *"it puked because it had bad data in all these records."* **Why it lands:** it's fast, read-only, and produces a genuinely startling artifact from their own system. It also proves the point behind A4: this is what's blocking the scheduling automation.

B5. Field impact analysis — "what breaks if I touch this?" — ★ *highest emotional payoff for Mike.* Pick a field or file. Get the complete blast radius: every program, display file, report and interface touching it. Two minutes. **Why it lands:** this is a direct answer to *"it's breaking, breaking, breaking, breaking — oh, we didn't know it affected that."* It's the fear that stops Mike changing anything, resolved in front of him. If only one live demo works, make it this one.

B6. Duplicate part number scan, live. Run the A7 analysis live against the item master. Julia's top complaint, answered with their data while she watches.

B7. The eight-screens diff. Auto-diff the eight variants of the 90-option menu screen. What's genuinely different, what's accidental drift, what could be one role-driven screen.

B8. Interface and integration map. Every touchpoint to ADP, TimeClock Plus, Teamcenter, and the cloud timekeeping system — what data moves, which way, on what trigger. Nobody currently has this picture.

B9. Journal-based audit — who changed what. Answers "which users are entering invalid data, through which programs" — the input side of the guardrails conversation.

Change and build — proves it ships, not just analyses

B10. Add a column to a subfile. Gary's proven, reliable demo. Run it on a **Fletcher** screen. Low risk, known timing — good opener.

B11. Green screen → EJS conversion, live. Convert one Fletcher screen. Ideally a parts or machine-record screen so it visually rhymes with the A1 and A2 mockups — the moment where *"this is what we built for you"* and *"and here's it happening to your real screen"* connect.

B12. RPG II / fixed-format → free-format + subprocedures. — ★ *aimed squarely at Mike.* Convert one legacy program, with an **A/B behavioural comparison** proving identical output. **Why it lands:** Mike is *already doing this by hand* as his personal technical-debt strategy, and James fixed the three-year bug precisely by breaking a program into subprocedures. This isn't a foreign idea we're selling — it's **their own method, at speed**. Hard to feel threatened by a tool doing more of the thing you already believe in.

B13. Ad-hoc report on demand. Someone in the room asks for a report the way they'd normally ask Odom. Deliver it live, then schedule it as a daily PDF. **Best done with a non-IT person asking** — Julia, or Chuck. (*Julia's own observation: her highest-value AI users are her most senior people, because they ask better questions. Demo it with one.*)

B14. Bug → reproduce → fix → test. Take a real open issue. Show the full cycle including the self-built test plan.

B15. Build the "Fletcher Coding Standards" skill, live. Point it at their six best programs, generate a standards skill, then show it linting a new change. Gary described this; doing it live with **their** code makes it concrete — and it's a direct answer to Mike's real anxiety, because it's how his judgement outlives his tenure.

B16. Spool file / green-bar report → browser + Excel. Small, visual, instantly understood by non-technical attendees.

B17. Memory / retrospective. Show CoderFlow recording a Fletcher-specific lesson and applying it on the next task. Speaks to *"does this get better or do we start from zero every time?"*

Two set-pieces that aren't features

B18. The controlled failure. — ★ *do not skip.* Deliberately run a task that goes wrong. Show the diff being rejected, the container deleted, the temp library cleaned up, nothing left behind. **Why it matters:** the last demo was flawless and produced *fear*. Mike's real question is "what happens when it breaks." A clean recovery answers it; another clean success doesn't.

B19. Hand over the keyboard. — ★ *do not skip.* **Matthew Duncan** (CEO's son, Purdue CS) or **Nick Ruggles** launches a task themselves and sees it complete. Not watching — driving. **Why it matters:** Julia's explicit strategic ask is that Matthew is exposed and sold, because Rod will hear it from him. A junior person succeeding unaided is the most persuasive thing that can happen in the room, and it's the only demo where **we're not the ones performing**.

5. What to actually do — recommended shortlist

Exhaustive lists are for choosing from. If I had to pick:

Build ahead (four, ~2 weeks):

1. **A1 Parts Finder** — the business-understanding piece
2. **A2 Fleet 360** — the revenue piece, for Chuck and Rod
3. **A7 Duplicate Part Analysis** — the credibility piece, real data (*chase the extract now*)
4. **A6 Warehouse scan fix** — the liability (*needs their hardware — longest lead time*)

Then **A4 Schedule Board** if time allows; it's the most visually striking and the most visible manual process in the building.

Run live (six, in this order):

1. **B10** add a column — safe opener, known timing
2. **B5** field impact analysis — Mike's fear, resolved
3. **B4** database health scan — the startling artifact
4. **B12** RPG II → free-format — Mike's own method at speed
5. **B18** the controlled failure
6. **B19** Matthew drives

Long-runners (**B1**, **B2**, **B3**) launch at the start and get reviewed as they land, the way Gary paced it last time.

Suggested run of show

Slot	Content	For
Open	Launch B1/B2/B3 in the background. No slides.	—
1	A1 + A2 — clearly labelled <i>"mockups of your business, built this week"</i>	Julia, Chuck, Rod
2	B5 then B4 — live, their data, their fear	Mike, Ray
3	B12 — their own modernization method, at speed	Mike
4	Review B2 documentation output → hand Mike the red pen	Mike
5	B18 controlled failure + teardown	Mike, Ray
6	B19 Matthew drives a task	Rod (via Matthew)
7	Security & data-egress brief — unprompted	Ray
Close	The one-pager: Infor Phase 1 vs. Profound Year 1	Julia, Chuck, Rod

6. Effort, dependency and risk

#	Demo	Effort	Needs from Fletcher	Risk if we skip
A1	Parts Finder	M	— (better with item master)	Miss the "understands us" moment
A2	Fleet 360	M	—	No revenue story for Rod/Chuck
A3	Order→Eng→BOM	M-L	Order flow sketch	Julia's #1 ask unanswered
A4	Schedule Board	M	Work-order extract ideal	Miss the most visible manual process
A5	Field Service tablet	M	Tablet spec, checklist	—
A6	Warehouse scan fix	S-M	⚠ Hardware + label + program	Mike's objection stands unanswered
A7	Duplicate part analysis	S-M	⚠ Item-master extract	Lose the highest-credibility artifact

#	Demo	Effort	Needs from Fletcher	Risk if we skip
A8	Exec dashboard	S-M	—	—
A9	MSHA dossier	S-M	How they track certs today	—
A10	Warranty triage	S	—	—
A11	Distributor portal	M	—	—
A12	Cannon reconciliation	S-M	⚠ Verify it's real first	Asserting a problem they don't have
B1- B17	Live runs	—	⚠ Source repo + read-only DB	Fall back to generic data — repeats the mistake

The three things to request today, because we don't control their lead time:

1. **Source code into a repo** — gates nearly all of Tier B.
2. **Read-only DB access / an item-master extract** — gates A7 and B4/B6, the highest-credibility items.
3. **A tablet, a sample label, and the scan program** — gates A6, and physical hardware can't be rushed.

7. Two warnings

Label the mocks. Ray already spotted invented fleet data once and asked whether it came off their website. Say "this is a mockup with invented data" *before* anyone asks, every time. The credit we get for volunteering it is what makes the un-caveated real-data artifacts believable.

Don't let the POCs imply we've already scoped the work. Julia is trying to sell a **one-year defined engagement** internally, and Rod's instinct will be to read polished mockups as either "they've already decided what we need" or "this is nearly done, why is it a year?" Frame every Tier A build as *"a week's work to show you what's possible — not a plan, and not an estimate."*

8. Open questions this raises for the customer

Vertical-specific, and worth asking before or during the onsite:

1. Where does **configuration and BOM** for a custom machine actually live — Teamcenter, the IBM i, or both? Which is authoritative?
2. How does a parts rep **identify a part** for a 20-year-old machine today? What do they search, and what fails?
3. Is there **serial-number-level machine history** — build config, service events, modernization, certifications — and is it queryable, or is it paper and tribal knowledge?
4. How is **modernization candidacy** identified today? Customer request, or does Fletcher proactively target it? (*If reactive, that's quantifiable revenue left on the table — the strongest CFO argument in the whole deck.*)
5. Is **Cannon Mining** product data integrated with Fletcher's, or parallel?
6. How are **MSHA approvals and certification tags** tracked, and where?
7. Do **distributors** have any system access, or is it all phone and email?

8. What drives the **production schedule** besides the sticky notes — what feeds it, who owns it?
9. Which system holds the **installed base** — who owns which machine, at which mine?
10. Does **Engineered Solutions** (non-mining) run on the same system and part numbering as mining?